
Multimodal Failure Prognosis for an Autonomous UAV Delivery Fleet

A tri-modal predictive-maintenance challenge
(tabular telemetry + raw onboard signals + free-text maintenance notes)

One-line summary. For every drone flight, estimate the probability that the aircraft will suffer a *critical powertrain failure within its next five flights*, by fusing structured telemetry, raw multivariate sensor streams, and unstructured technician notes. The predictive evidence is split across the modalities and *interacts*; teams that treat the three sources as three separate problems will not place.

Contents

1	Overview	2
2	Background and Motivation	2
3	Problem Formulation	2
3.1	Data and notation	2
3.2	Prediction target	3
3.3	Grouped structure (read carefully)	3
4	Dataset	3
4.1	Files and join key	3
4.2	Tabular modality	3
4.3	Signal modality	4
4.4	Text modality	4
4.5	Scale and splits	5
4.6	Auxiliary training label	5
4.7	Data-realism caveats	5
5	Evaluation	5
5.1	Primary metric: Average Precision (AUPRC)	5
5.2	Why AUPRC	5
6	Submission Specification	5
7	Rules and Constraints	6
8	Suggested Workflow	6
9	Deliverables	6
	Appendix A. Loading the data	6

1 Overview

SkyGuard Logistics runs one of the region’s largest autonomous last-mile delivery fleets: several thousand drones that take off thousands of times a day, ferrying medical samples, spare parts, and parcels across dense cities and remote valleys alike. For years its maintenance philosophy was simple and unforgiving: fly each aircraft until something breaks, then repair it.

That approach held until the morning a delivery drone lost a motor two minutes after takeoff and dropped a cooler of vaccines into a river. The incident review reached an uncomfortable conclusion. The warning signs had been accumulating for over a week, scattered across vibration traces, battery logs, and a technician’s offhand note about “a faint grinding on spin-up” that nobody had the time to connect.

Out of that near-miss came Project SkyGuard. Its mandate is to move the fleet from **reactive** repair to **predictive** maintenance: to flag an aircraft for inspection *before* it fails, while it is still safely on the ground, using nothing but the operational history the fleet already records. You have been handed that history.

Each record in the dataset is one completed flight, and it bundles three very different views of the same aircraft: structured telemetry, the raw onboard sensor streams, and the maintenance technician’s free-text note. Your job is to turn those three views into a single calibrated number for every flight, the probability that this aircraft is about to fail. Submissions are ranked on a hidden test set by Average Precision.

2 Background and Motivation

Drones fail the way most machines do: not all at once, but through a slow accumulation of wear that leaves a trail long before the final failure. The trail is real, but it is faint, and it is fragmented across instruments that were never designed to be read together:

- **Mechanical wear** (e.g. motor-bearing fatigue) raises structural vibration and alters the spectral content of accelerometer and dedicated vibration channels.
- **Battery degradation** manifests as voltage sag under load and increased current ripple, visible on the power-bus and motor-current channels.
- **Human observation** is captured in free-text post-flight notes, which are information-rich but inconsistent, incomplete, and occasionally misleading.

No single channel tells the whole story. A vibration pattern that is benign on one airframe is alarming on another; a note may flag a fault the sensors barely register, or cry wolf when nothing is wrong. The real task here is not to read any one instrument well, but to fuse all three into a judgement no single source could support on its own. That is the engineering problem this challenge is built around.

3 Problem Formulation

3.1 Data and notation

The dataset is a collection of flight records

$$\mathcal{D} = \{ (\mathbf{x}_i, \mathbf{S}_i, \tau_i, y_i) \}_{i=1}^N,$$

where for the i -th flight:

- $\mathbf{x}_i \in \mathcal{X}$ is the **tabular** feature vector (mixed numeric and categorical: airframe configuration + operating conditions + cumulative usage);
- $\mathbf{S}_i \in \mathbb{R}^{128 \times 8}$ is the **signal** tensor (128 time-steps $\times$ 8 channels of resampled onboard telemetry);
- $\tau_i \in \mathcal{T}$ is the **text** note (a possibly empty token sequence);
- $y_i \in \{0, 1\}$ is the binary target (training set only).

Each flight belongs to a specific aircraft $g_i \in \mathcal{G}$ and carries a within-aircraft flight index $k_i \in \{1, 2, \dots\}$.

3.2 Prediction target

Let aircraft g experience its (single, terminal) critical failure at flight index T_g (set $T_g = \infty$ if it never fails within the observation window). With horizon $H = 5$, the label attached to flight i is

$$y_i = \mathbb{1}[1 \leq T_{g_i} - k_i \leq H], \quad H = 5. \quad (1)$$

In words: $y_i = 1$ iff the aircraft that flew flight i fails on one of its *next five* flights. This is a forward-looking prognosis from the lead-up to failure, not detection of a failure that has already occurred.

Objective

Learn a scoring function

$$f_{\theta} : \mathcal{X} \times \mathbb{R}^{128 \times 8} \times \mathcal{T} \longrightarrow [0, 1], \quad \hat{p}_i = f_{\theta}(\mathbf{x}_i, \mathbf{S}_i, \tau_i) \approx \Pr(y_i = 1 | \mathbf{x}_i, \mathbf{S}_i, \tau_i),$$

that ranks soon-to-fail flights above healthy ones. The positive class is **rare** ($\Pr(y = 1) \approx 0.12$).

3.3 Grouped structure (read carefully)

Flights of the same aircraft are statistically dependent: they share a slowly varying health state. The official train/test partition is therefore made **at the aircraft level**:

$$\mathcal{G} = \mathcal{G}_{\text{train}} \sqcup \mathcal{G}_{\text{test}}, \quad \mathcal{G}_{\text{train}} \cap \mathcal{G}_{\text{test}} = \emptyset.$$

No aircraft appears in both splits. You are strongly advised to mirror this structure in your own cross-validation (group k -fold by `drone_id`), otherwise your internal estimates may be optimistic.

4 Dataset

4.1 Files and join key

All three modalities are aligned on the integer key `flight_id`. One row of the tabular file, one slice of the signal tensor, and one note correspond to the same flight.

```
data/
|-- train/  train_tabular.csv  train_notes.csv  train_signals.npz  (labelled)
|-- test/   test_tabular.csv   test_notes.csv   test_signals.npz   (unlabelled)
'-- sample_submission.csv
```

4.2 Tabular modality

One row per flight. Airframe configuration is constant within an aircraft; operating conditions and cumulative-usage fields vary per flight.

Field	Type	Description
flight_id	int	Unique flight key (join across modalities).
drone_id	str	Aircraft identifier; defines the grouped split.
model	cat	Airframe model (affects structural resonance).
motor_type	cat	Motor variant; differing wear and sensor response.
firmware_version	cat	Flight-controller firmware.
battery_capacity_mAh	int	Installed pack capacity.
max_payload_g	float	Rated maximum payload.
propeller_in	float	Propeller diameter (in).
manufacture_batch	int	Production batch index.
operator_region	cat	Operating region.
flight_index	int	Sequence number of this flight in the aircraft's life.
payload_g	float	Payload carried this flight.
ambient_temp_C	float	Takeoff ambient temperature.
wind_speed_ms	float	Mean wind speed.
flight_duration_min	float	Flight duration.
avg_throttle	float	Mean throttle (0–1).
num_aggressive_maneuvers	int	Count of high-load maneuvers.
cumulative_flight_hours	float	Airframe hours to date.
battery_cycles	int	Charge cycles on the current pack.
failure_mode	cat	Train only. Dominant failure path (bearing/battery/none); auxiliary.
failure_within_horizon	int	Train only. Target.

4.3 Signal modality

The file `*_signals.npz` (load with `numpy.load`) contains `flight_id` (N), `signals` ($N, 128, 8$), and `channel_names` (8). The signal array is row-aligned to its `flight_id` vector.

Idx	Channel	Source
0–2	accel_x/y/z	IMU acceleration
3	gyro_z	Yaw-rate gyro
4–5	motor_current_1/2	Per-motor current draw
6	vibration	Dedicated structural vibration sensor
7	battery_voltage	Power-bus voltage (normalised)

Sensor dropout. On a minority of flights one or more channels are zeroed over a contiguous span (a real recorder artefact). Robust pipelines must tolerate partially missing channels rather than assuming every channel is always present.

4.4 Text modality

`*_notes.csv` maps `flight_id` to a short free-text `maintenance_note`. Roughly **28%** of notes are empty strings. Notes range from routine sign-offs to explicit fault flags, in inconsistent phrasing, and a small fraction are misleading.

4.5 Scale and splits

Split	Flights	Aircraft	Labels
Train	$\approx 15,900$	≈ 620	provided
Test	$\approx 7,500$	≈ 290	withheld

4.6 Auxiliary training label

The train-only field `failure_mode` records the dominant degradation path for labelled failures (bearing, battery, or none). You may use it freely during training (for example as an auxiliary multi-task target), but it is **not available at test time**, so it cannot be a model input.

4.7 Data-realism caveats

This is operational data and behaves like it:

1. **Modalities go missing** (empty notes; sensor dropout). Discarding incomplete records throws away signal, and positives with it.
2. **The fleet is heterogeneous**. A given sensor pattern does not carry the same meaning across airframe models and motor variants.
3. **Train and test fleets differ**. Some hardware configurations are under-represented or absent in training; a model that memorises the training fleet will not generalise.
4. **Notes are noisy**, and **a small fraction of labels are mislabelled**, as in any real maintenance log.

5 Evaluation

5.1 Primary metric: Average Precision (AUPRC)

Submissions are ranked by the area under the precision–recall curve, computed from the predicted scores. At a decision threshold t , with $TP(t)$, $FP(t)$, $FN(t)$ the usual counts,

$$\text{Precision}(t) = \frac{TP(t)}{TP(t) + FP(t)}, \quad \text{Recall}(t) = \frac{TP(t)}{TP(t) + FN(t)}.$$

Average Precision summarises the curve as a threshold-free score,

$$AP = \sum_n (R_n - R_{n-1}) P_n, \quad (2)$$

where (P_n, R_n) are precision/recall at the n -th distinct threshold (ordering predictions by score). Higher is better; a random ranking scores $AP \approx \Pr(y = 1)$.

5.2 Why AUPRC

Under heavy class imbalance, ROC–AUC is dominated by the abundant negatives and can look high for a weak detector. AUPRC focuses on the quality of the positive ranking, precisely what a maintenance-triage system cares about, and requires no threshold to be chosen in advance.

6 Submission Specification

A CSV with one row per test flight:

```
flight_id,failure_within_horizon
30001,0.0123
30002,0.8841
...
```

Column	Type	Constraint
flight_id	int	Must cover every test flight exactly once.
failure_within_horizon	float	Probability/score in $[0, 1]$ (out-of-range values are clipped).

A submission missing any test `flight_id` is rejected. See `sample_submission.csv`.

7 Rules and Constraints

- Any modelling approach is permitted: classical ML, deep learning, or hybrids. No model family is privileged.
- **No external data** and **no manual labelling** of the test set. Use only the provided files.
- Pretrained open-source models / text encoders are allowed if disclosed in the writeup.
- Submissions must be reproducible from the submitted code.

8 Suggested Workflow

Not prescriptive, but a sensible path:

1. Establish per-modality baselines (tabular GBM; signal features; text classifier) before fusing, so you know what each modality contributes alone.
2. Engineer signal representations that do not hard-code assumptions about a fixed fleet (e.g. full-spectrum or learned features rather than a single hand-picked band).
3. Let modalities *interact*; the marginal use of a sensor feature can hide structure that only appears when conditioned on the airframe configuration.
4. Handle missing notes and sensor dropout explicitly (presence indicators, imputation, or modality dropout during training).
5. Validate with **group k -fold by aircraft**; calibrate your probabilities.

9 Deliverables

Three artefacts:

1. `submission.csv`, covering the full test set in the format of Section 6;
2. a concise (≤ 2 page) methodology writeup and a clean, runnable notebook, describing how you fused the modalities, validated your model, and handled missing data;
3. a short presentation deck (5–10 slides) for the judging round, summarising your problem framing, approach, key results, and the design decisions that drove them.

Appendix A. Loading the data

```

import numpy as np, pandas as pd

tab = pd.read_csv("data/train/train_tabular.csv")    # tabular + label
notes = pd.read_csv("data/train/train_notes.csv")    # flight_id -> text (may be
empty)
z = np.load("data/train/train_signals.npz")
signals = z["signals"]    # (N, 128, 8)
signal_ids = z["flight_id"]    # aligned to signals
channels = z["channel_names"]

# join signals back to the tabular rows
import numpy as np
pos = pd.Series(np.arange(len(signal_ids)), index=signal_ids)
row_for_flight = lambda fid: signals[pos.loc[fid]]

y = tab["failure_within_horizon"]    # rare positive (~12%)

```

End of problem statement.